

Color Tracking

In this lesson, let's use OpenCV to track a red color block with the pan-tilt and display the recognition results through the live camera feed.

Before starting the game, please prepare a red block.

1. Recognition Procedure

First, subscribe to the topic messages published by the color recognition node to obtain color information. Obtain the RGB image from the camera and resize it. Apply Gaussian blur to the image and convert the color space from RGB to Lab.

Then, perform color recognition on the objects in the circle using color thresholding. Apply masking to parts of the image. Masking is used to globally or locally mask the processed image using selected images, graphics, or objects.

Afterward, perform opening and closing operations on the object images. Next, outline the largest object with a circle.

Opening operation: erode the image and then dilate it. It is used to eliminate small objects, smooth the shape boundary, and not change its area. It can remove small grain noise and break the adhesion between objects.

Erosion: Eliminate the boundary points of the object, causing the boundary to shrink inward, and can remove objects smaller than the structural element.

Dilation: Expand the boundary points of the object, merge all background points in contact with the object into the object, and expand the boundary outward. Then, feedback the recognition results on the feedback screen. After matching the target color, the center position of the target image is obtained.

Finally, by publishing a corresponding topic message, the servo is controlled to rotate, so that the pan-tilt servo can follow the target

movement.

```

110
111 def start_srv_callback(self, request, response):
112     self.get_logger().info('\033[1;32m%s\033[0m' % "start color track")
113     self.start = True
114     response.success = True
115     response.message = "start"
116     return response
117
118 def stop_srv_callback(self, request, response):
119     self.get_logger().info('\033[1;32m%s\033[0m' % "stop color track")
120     self.start = False
121     res = self.send_request(ColorDetect.Request())
122     if res.success:
123         self.get_logger().info('\033[1;32m%s\033[0m' % 'set color success')
124     else:
125         self.get_logger().info('\033[1;32m%s\033[0m' % 'set color fail')
126     response.success = True
127     response.message = "stop"
128     return response
129
130 def get_color_callback(self, msg):
131     if msg.data != []:
132         if msg.data[0].radius > 10:
133             self.center = msg.data[0]
134         else:
135             self.center = None
136     else:
137         self.center = None

```

2. Operation Steps

Note:

1.The input command should be case sensitive, and keywords can be complemented using Tab key.

2.This game is only applicable to 2D pan-tilt monocular camera version.

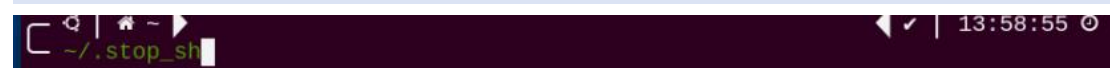
1) Start the robot and connect it to the remote control software VNC.



2) Click on  on the system desktop to open the command line terminal.

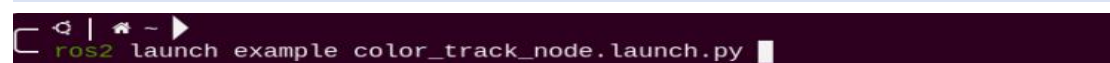
3) Run the command to stop the automatic startup service of the app.

```
~/stop_sh
```



4) Enter the command to start the game.

```
ros2 launch example color_track_node.launch.py
```



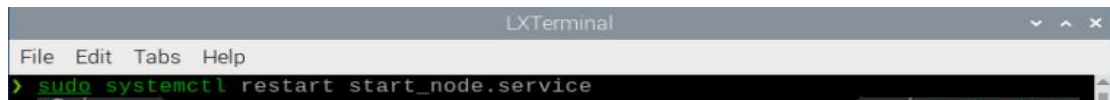
5) If you want to exit the game, press "Ctrl+C" in the terminal interface.

After experiencing the game, you can enable the app service through commands or by restarting the robot. If the app is not enabled, the related app functions will not work. If the robot is restarted, the app will be automatically enabled.



Click  and enter the command. Press enter to start the app, and wait for the buzzer to beep.

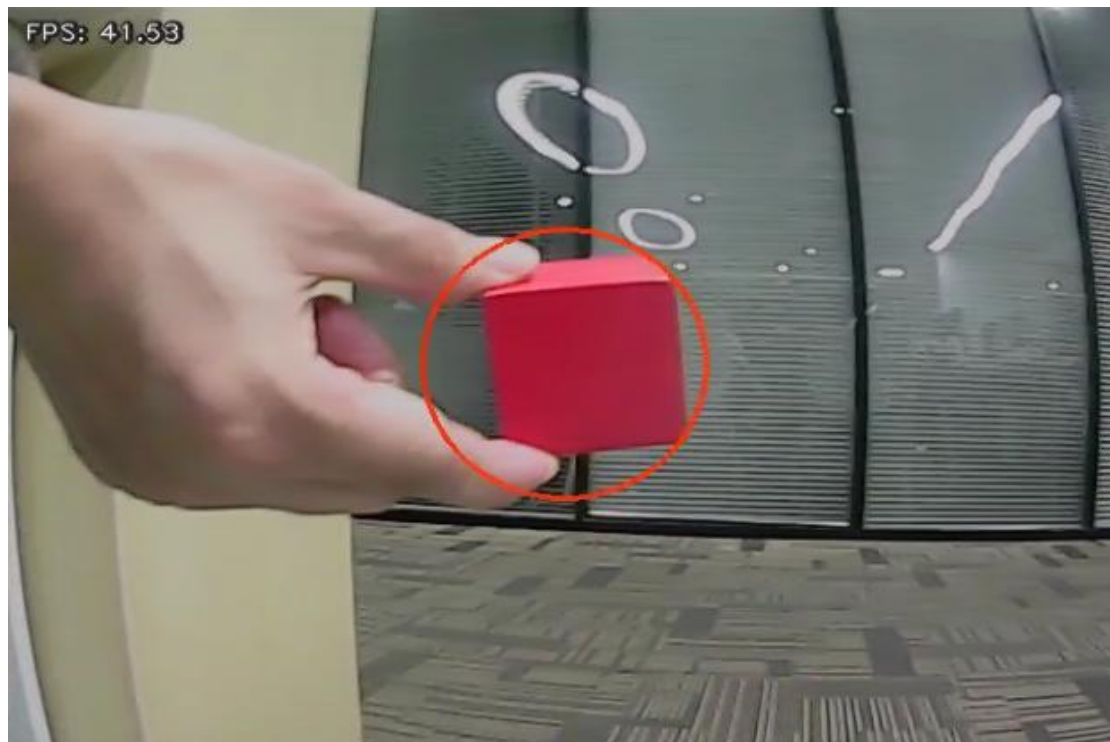
```
sudo systemctl restart start_node.service
```



3. Program Outcome

Note: After starting the game, please ensure that there are no other objects containing the recognized color in the field of view of the camera, as this may affect the affect the program outcome.

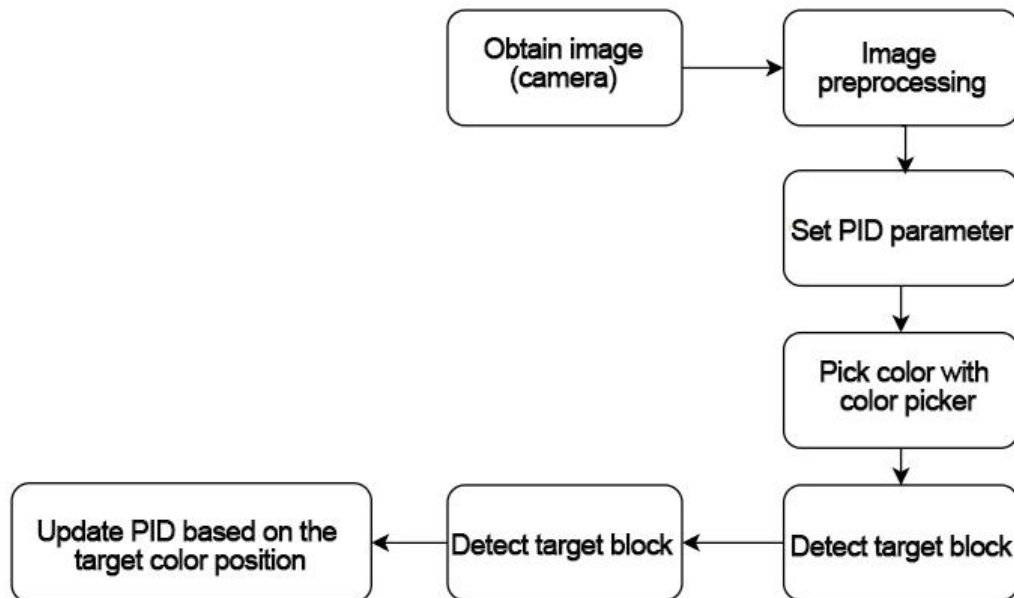
After starting the program, place the target object within the camera's field of view. Once the target object is recognized, it will be highlighted with a circle and the pan-tilt will track it.



4. Program Analysis

The source code of the program is located at:

`/home/ubuntu/ros2_ws/src/example/example/color_detect/color_detect_demo.py`



◆ Main function

```

168 def main():
169     node = ColorTrackNode('color_track')
170     executor = MultiThreadedExecutor()
171     executor.add_node(node)
172     executor.spin()
173     node.destroy_node()
174
175 if __name__ == "__main__":
176     main()
  
```

◆ pwm_controller

```

53 def pwm_controller(self, position_data):
54     pwm_list = []
55     msg = SetPWMServoState()
56     msg.duration = 0.2
57     for i in range(len(position_data)):
58         pos = PWMServoState()
59         pos.id = [i+1]
60         pos.position = [int(position_data[i])]
61         pwm_list.append(pos)
62     msg.state = pwm_list
63     self.pwm_pub.publish(msg)
  
```

Construct a PWM signal based on the position data received, and publish it to the robot controller.

◆ Set color threshold

```

95     def set_color_srv_callback(self, request, response):
96         self.get_logger().info('\033[1;32m%s\033[0m' % "set_color")
97         msg = SetColorDetectParam.Request()
98         msg_red = ColorDetect()
99         msg_red.color_name = request.data
100        msg_red.detect_type = 'circle'
101        msg.data = [msg_red]
102        res = self.send_request(self.set_color_client, msg)
103        if res.success:
104            self.get_logger().info('\033[1;32m%s\033[0m' % 'start_track_%s'%msg_red.color_name)
105        else:
106            self.get_logger().info('\033[1;32m%s\033[0m' % 'track_fail')
107        response.success = True
108        response.message = "set_color"
109        return response
110

```

Set the color threshold. Set the color detection parameters based on the request and publish them.

◆ Adjust the position of servo

```

140     while self.running:
141         if self.center is not None and self.start:
142             t1 = time.time()
143             self.pid_y.SetPoint = self.center.width / 2
144             self.pid_y.update(self.center.width - self.center.x)
145             self.y_dis -= self.pid_y.output
146             if self.y_dis < 800:
147                 self.y_dis = 800
148             if self.y_dis > 1900:
149                 self.y_dis = 1900
150             self.pid_z.SetPoint = self.center.height / 2
151             self.pid_z.update(self.center.y)
152             self.z_dis -= self.pid_z.output
153             if self.z_dis > 1900:
154                 self.z_dis = 1900
155             if self.z_dis < 800:
156                 self.z_dis = 800
157             self.pwm_controller([self.z_dis, self.y_dis])
158             t2 = time.time()
159             t = t2 - t1

```

Check if the color has been detected. If detected, set it to True and record the current time (t1) to control the frequency of the loop execution.

PID control: two PID controllers, “pid_y” and “pid_z”, are used to adjust the position of the pan-tilt in the horizontal (y-axis) and vertical (z-axis) directions.

Position limitation: Limit the adjusted position values “self.y_dis” and “self.z_dis”. Send the adjusted position values to the pan-tilt to control it to move to the new position.